OPERATING SYSTEMS – ASSIGNMENT 1

Name: S. Sharvesh **Register No:** 192525458

TASK 3 – Evaluate Operating-System Structures (4 Marks)

Evaluate simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures.

Discuss reliability, performance and ease of updating; which structure suits patient monitors and database server; and how Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured.

Alternative Concept and Complexity Analysis: If bedside monitors used a monolithic kernel instead of a microkernel, evaluate service-request time, kernel memory and the effect of a faulty driver.

Evidence Expected: A comparison of structures and a defended choice for two environments.

ANSWER

3.1 Structure selection for two environments

Bedside monitors: choose a microkernel-based real-time OS such as QNX, provided the exact hardware platform and configured system pass worst-case timing validation. The small trusted kernel and user-space drivers/services reduce the effect of a faulty non-kernel component. The microkernel must still meet the 10 ms end-to-end alarm deadline after sensor input, scheduling, processing and alarm output.

Database server: choose a modular Linux kernel. Linux is a monolithic kernel with loadable kernel modules; it offers mature storage, networking, multiprocessor and database support. Its modularity makes driver and feature management practical, while clustering and database replication provide availability. A microkernel is not automatically more available than a correctly engineered Linux cluster.

3.2 Examples: Linux, Windows, Android and QNX/FreeRTOS

Linux: monolithic but modular kernel; loadable modules support drivers and filesystems.

Windows: commonly described as a hybrid kernel architecture, with executive services, a kernel layer and a hardware abstraction layer.

Android: uses the Linux kernel with Android framework and runtime layers; applications are sandboxed and use managed services.

QNX: microkernel-based RTOS; many services and drivers execute outside the small kernel.

FreeRTOS: a small real-time kernel intended for embedded systems; it is not a full desktop OS and many services are supplied by the application or optional components.

3.3 Alternative: monolithic kernel on bedside monitors

A monolithic kernel can provide a faster service path because many services are invoked through a system call and execute inside kernel space without inter-process message passing. A microkernel service request may require IPC/message passing, scheduling and context switches; therefore it can have higher constant-factor latency. Both can have O(1) or O(n) algorithms depending on the service, so Big-O alone does not determine latency.

A monolithic kernel may occupy more kernel memory because more services and drivers execute in privileged space. A microkernel aims to keep the privileged core small, although user-space servers still consume memory. The most important difference is fault containment: a faulty monolithic driver can corrupt kernel memory and crash the entire monitor, whereas a user-space driver in a microkernel may be restarted without crashing the kernel. For a patient monitor, fault isolation is more important than saving a small amount of IPC overhead, provided measured worst-case latency remains within 10 ms.

QUESTION FROM THE PROVIDED ASSIGNMENT

Structure | Reliability | Performance | Updating and suitability

Simple / monolithic | Many services run in kernel space; a faulty driver can crash the whole OS. | Fast internal calls and low communication overhead. | Harder to isolate and update components safely. Not the preferred monitor

design.

Layered | Clear separation can limit dependency errors. | Layer crossings may add overhead. | Easier to understand and test layer by layer; useful as a design principle.

Microkernel | Small kernel keeps essential functions; many drivers/services run in user space, improving fault isolation. | Message passing and context switches can add latency. | Services can be restarted or updated independently. Strong fit for safety-focused monitors if the selected RTOS meets timing requirements.

Modular kernel | Core kernel with loadable modules; a faulty kernel module can still affect the kernel. | Near-monolithic performance for in-kernel modules. | Drivers and features can be loaded/updated without rebuilding the whole kernel. Common in Linux.

Hybrid | Combines microkernel ideas with many services in kernel mode; reliability depends on implementation. | Can reduce message-passing overhead. | Practical balance of performance and modularity; common in desktop/mobile systems.